Especially on time-critical components or in data-intensive applications, we often need to understand the internals of the data structures, and the details of its implementation, to make sure we can integrate it in our solution without introducing a bottle neck[4].

Every abstraction needs to be implemented using a concrete data structure: for example, a stack can be implemented using a list, an array, in theory even a heap (although it would be silly, as it will be clear in a few sections). The choice of the underlying data structure used to implement a container will only influence the container's performance. Choosing the best implementation is usually a tradeoff: some data structures allow to speed-up some operations, but will make other operations slower.

2.5.1 Comparing performance

For the implementation of a priority queue, we will initially consider three naïve alternatives using core data structures discussed in appendix C: an unsorted array, where we just add elements to its end, a sorted array, where we make sure the ordering is reinstated every time we add a new element, and balanced trees, of whom heaps are a special case. Let's compare, in table 2.2, the running times[5] for basic operations implemented with these data structures.

2.5.2 What’s the right concrete data structure?